

CHAGent: Context-aware Human-centric Access Control Policy Generation

Anonymous Authors

Abstract—Manually generating access control policies from high-level requirement specifications is laborious and error-prone. Existing automated solutions lack context awareness and do not support identifying and recovering from generation errors, limiting their effectiveness in practice. We present CHAGent, a novel automated access control policy generation framework. CHAGent facilitates human involvement in the policy generation, while automating its laborious aspects. Our implementation of CHAGent translates natural language access requirements into context-aware policies, with an F1 score of 81.4%, significantly outperforming existing policy generation solutions. CHAGent’s automated validation and refinement mechanism improves the performance by 4.2%. If automated refinement fails, a novel error locator and explainable feedback guide human-in-the-loop refinement. A user study with policy implementers confirmed that CHAGent’s context awareness enables accurate policy generation, while its validation, error localization, and feedback help improve policy accuracy and trust. We also introduce three annotated datasets to support future research and address data scarcity in the domain.

Index Terms—access control, natural language processing

1. Introduction

In organizations, access control failures can occur when access control policies are manually implemented/generated from the organization’s high-level requirement specifications by policy implementers [1]–[4]. These specifications are written by policy makers (distinct from policy implementers) [3], explaining how information access should be managed within the organization [5]. However, they are unstructured and ambiguous, making it difficult for policy implementers to understand [2], [5]. Consequently, manually generating policies becomes laborious and error-prone [1], [5], [6].

To help policy implementers generate access control policies, previous research has introduced automated access control policy generation solutions [4], [5], [7]–[17]. They automatically process high-level requirement specification documents, identify access requirements/Natural Language Access Control Policies (NLACPs), and extract policy components like subjects/roles, actions, and objects/resources for building access control policies from them, using machine learning (ML) and natural language processing (NLP) techniques [4], [7], [14], [18]. By doing so, they attempted to reduce access control failures due to human mistakes [19].

However, the lack of support for identifying and recovering from automated policy generation errors, and the

lack of awareness of the organization context, significantly undermine the performance of existing automated policy generation solutions [1], [4]–[6], [20]. These solutions remove the human factor entirely from the automated policy generation process. This poses security risks, as automated policy generation may produce incorrect policies due to the inherent limitations of the NLP models, such as hallucinations [1]. Since there has been no mechanism to identify such policy generation errors and involve policy implementers to refine them accurately, existing systems face a high risk of applying incorrect policies to the authorization system, causing access control failures [6].

Moreover, most existing automated policy generation systems rely on NLP models [4], [5], [7], which are not adapted to the access control policy context. As a result, they often extract incorrect policy components, resulting in low policy generation accuracy [4], [7]. Even if they are adapted via datasets, once deployed in an organization, they lack awareness of the organizational context (e.g., roles, actions, and resources). This leads the systems to generate incorrect policies that do not align with the organization’s authorization system.

We present CHAGent, a context-aware, human-centric access control policy generation framework. Unlike existing approaches, CHAGent automates labor-intensive tasks, with high accuracy and minimal manual effort. At the same time, CHAGent can detect the majority of generation errors and actively engage human policy implementers to recover from them. To this end, we introduce a novel policy validation, refinement, error localization, and explainable feedback loop for access control policy generation. Here, validation refers to checking whether a generated policy accurately represents the NLACP, distinct from formal policy verification [21].

To reduce risks of data leakages from using third-party models for confidential policy generation [22], [23], CHAGent is designed to be implemented using locally deployable Small Language Models (SLMs). To enhance accuracy, CHAGent incorporates both access control and organizational context awareness, enabling it to translate nuanced NLACPs into policies aligned with the organization’s authorization system. Specifically, CHAGent is adapted to access control policy context and organizational context without costly training. It incorporates automated validation and refinement to correct generation errors. If automated refinement fails, CHAGent localizes the error and provides explainable feedback to support human-in-the-loop recovery.

This paper makes the following contributions.

- We develop CHAGent, a novel Context-aware

Human-centric Access control policy Generation framework to translate NLACPs into *context-aware* access control policies (Section 3).

- We introduce access control-specific structured information extraction that extracts word-level information and information hidden in the NLACP’s meaning when generating policies (Section 3).
- We introduce a novel policy validation-refinement and error localization mechanisms, facilitating automatic and human-in-the-loop refinement via feedback based on explainable security (Section 3).
- We implement CHAGent using SLMs and evaluate it through real-world high-level requirement specifications [14], [18]. We demonstrate that it identifies NLACPs and generates policies with subjects, actions, resources, purpose, and conditions, achieving an F1 score of 81.4%, significantly outperforming the current state of the art [7] and even larger generic black-box models (Section 5).
- We evaluated CHAGent’s implementation, called CHAGentSys, with 17 professional policy implementers. We found that it helps implementers effectively generate complex context-aware policies, and that its validation and human-in-the-loop refinement guides them to refine incorrect policies accurately. This builds their trust in CHAGentSys and motivates its adoption (Section 6).
- We introduce three datasets to address data scarcity in the domain (Section 4).

2. Related Work

Prior work has proposed numerous systems [5], [7], [10], [14], [17], [18], [25], [26] to identify NLACPs from requirement documents and extract their policy components. This is known as *top-down policy engineering* [8], distinct from mining access control policies from sources like logs, known as *bottom-up policy engineering* [27], [28]. Text2Policy [18] uses semantic pattern matching for NLACP identification and shallow parsing for component extraction. However, its limited semantic patterns hinder extraction from complex NLACPs. To address this, Slankas et al. proposed ACRE [14]–[16], [29], which applies k-Nearest Neighbors for NLACP identification and dependency parsing with bootstrapping [30] to extract Access Control Rules (ACRs) and their components [14]. Despite bootstrapping enabling dynamic pattern expansion, ACRE depends on repetitive sentence structures and struggles with complex NLACPs [5].

To improve the accuracy of policy generation without relying on limited syntactic rules that restrict the systems’ awareness of access control policy context [6], deep-learning-based access control policy generation systems were developed. For instance, SR2ACM [31] and LitroACP [17] employed BERT-based named entity recognition (NER) to extract policy components from NLACPs. However, NER focuses solely on identifying components, failing to capture relationships among them. As a result, it cannot determine

who performed what action on which resource when multiple ACRs are present in an NLACP [6]. Since these relationships are crucial for access control policies [14], NER-based approaches fall short for accurate policy generation [6].

Therefore, Narouei et al. [5] and Xia et al. [7] used deep learning-based SRL models to extract policy components of ACRs with their relationships. Recent systems [24], [25] leverage proprietary large LMs (LLMs) like GPT-4 to translate access requirements into access control policies. However, they introduce security risks like data leakages, since confidential access requirements must be shared with third parties that own the LLM for translation. Moreover, because the organization does not control third-party models, provider-side changes may affect policy generation performance [23]. Nevertheless, these systems lack access control policy context awareness due to scarce domain datasets [7], [26], causing inaccurate policy component extraction and low accuracy [6]. Even when adapted using domain datasets, they remain unaware of organization-specific roles and resources, potentially generating policies misaligned with the organization’s authorization system. While retraining on organization-specific data can mitigate this, it is costly [32] and must be repeated with every organizational change [6].

Moreover, all existing systems share common shortcomings due to the lack of access control policy and organizational context awareness. Notably, they fail to handle complex NLACPs containing multiple Access Control Rules (ACRs) with different access decisions [7], [14], [18], instead assigning a single overall decision to the entire NLACP (e.g., “*The [nurse cannot read records]_{DENY}, but the [doctor can]_{ALLOW}*”). This results in incorrect policies when conflicting access decisions coexist within an NLACP.

Even with access control and organizational context awareness, automated policy generation systems remain insecure without human involvement [1], [19], as NLP/AI errors can produce incorrect policies that cause access control failures. Therefore, a human-in-the-loop refinement step is essential before deploying generated policies [1], [6]. Yet none of the existing solutions facilitates this step, leaving a critical gap in research on access control policy generation.

As shown in Table 1, CHAGent addresses the identified research gaps. It is adapted to the access control policy context and leverages organizational context to generate accurate policies. CHAGent can be implemented using locally deployable SLMs, preventing confidential access requirements from being shared with third-party LLMs [23]. It validates and iteratively refines generated policies, and when a policy is too complex to refine automatically, it automatically locates the error and facilitates human-in-the-loop refinement through explainable feedback.

3. CHAGent

CHAGent’s high-level architecture is shown in Figure 1. It was developed around the following goals (DGs).

DG1 – Support Local Deployability: Sending confidential access requirements to third-party model providers risks exposing sensitive organizational structure and security posture

TABLE 1: Comparison of CHAGent to the existing policy generation approaches. AC: Access Control

	Underlying Technology	Local Deployability	AC Context Adaptation	Organizational Context Adaptation	Automated Validation	Automatic and human-in-the-loop refinement
Text2Policy [18]	Syntactic Parsing	✓	✗	✗	✗	✗
ACRE [14]	Syntactic Parsing	✓	✓	✗	✗	✗
Narouei et. al. [5]	SRL	✓	✗	✗	✗	✗
Xia et. al. [7]	SRL	✓	✗	✗	✗	✗
Lawal et. al. [24], Vatsa et al. [25]	LLM-based Generation	✗	✗	✗	✗	✗
CHAGent	LM-based Generation	✓	✓	✓	✓	✓

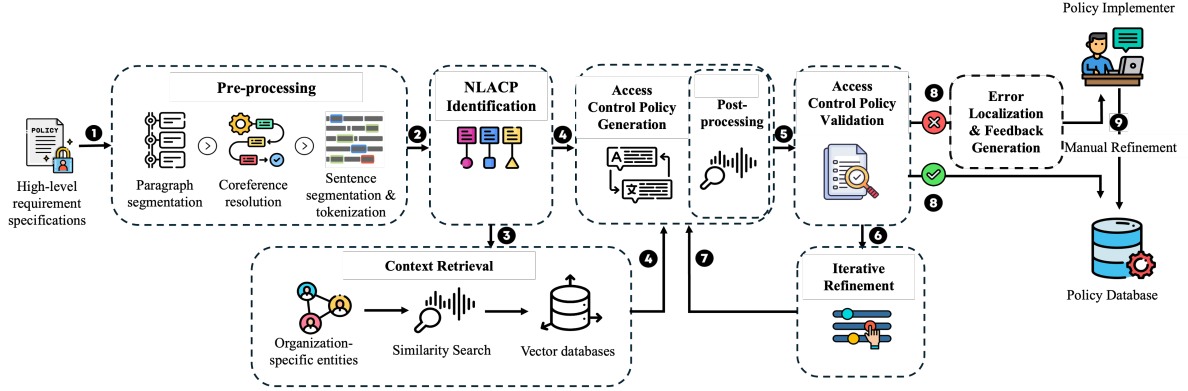


Figure 1: High-level architecture of CHAGent.

[22], [23]. Hence, CHAGent is designed to be implemented for local deployment without significantly compromising accuracy, keeping access requirements within the organization’s own infrastructure throughout policy generation.

DG2 – Context Awareness: Policies generated without access control and organizational context may not align with the organization’s authorization system, rendering them unenforceable (Sections 1 and 2). CHAGent adapts to both the access control policy context and the organizational context, enabling accurate translation of NLACPs.

DG3 – Automated Validation and Refinement: AI fallibility may occur due to hallucinations, resulting in incorrect policies, reducing automation accuracy [33]. Hence, CHAGent facilitates the identification of incorrect policies and iteratively refines them to improve the policy accuracy.

DG4 – Human-in-the-Loop: Removing humans from automated policy generation risks deploying incorrectly generated policies undetected [1]. CHAGent therefore involves policy implementers only when automated refinement fails, guiding them through explainable feedback to resolve it.

Next, we provide implementation details of CHAGent.

3.1. Pre-processing

Upon receiving a requirement specification document, CHAGent segments it into paragraphs and resolves coreferences [4], [14]. Coreference resolution identifies expressions referring to the same entity [5], enabling more accurate ACR extraction. Finally, CHAGent segments the paragraphs into sentences and tokenizes them to identify NLACPs.

3.2. NLACP Identification

High-level requirement specification documents contain both NLACPs and unrelated sentences, so NLACPs must be identified before policy translation [5], [7], [18]. We formulate this as a binary text classification task [4], [5], [7], [14]–[16], [29]. Accurate classification requires capturing full sentence context and word dependencies [34], unlike traditional ML techniques such as SVM used in prior work [5], [15], [18]. Therefore, CHAGent employs BERT [35], whose bidirectional nature captures context and word dependencies more effectively than traditional ML techniques as well as unidirectional GPT-like decoder-only LMs [34]. As a lightweight, open-source model, BERT also supports local deployment of CHAGent, satisfying **DG1**.

We first augment BERT (f_θ) with a feed-forward network (FFN) containing a single linear layer ($\mathbf{w}, \mathbf{b}$) on top of the $[CLS]$ token’s final hidden state, which provides an aggregate sentence s representation (P_θ) [35].

$$\mathbf{h}_{CLS} = f_\theta(s) \text{ and } \mathbf{z} = \mathbf{w}^T \mathbf{h}_{CLS} + \mathbf{b} \quad (1)$$

$$P_\theta(y | s) = \text{softmax}(\mathbf{z})_y = \frac{e^{z_y}}{e^{z_0} + e^{z_1}}, y \in \{0, 1\} \quad (2)$$

Then, we train P_θ with the access control policy (ACP) dataset ($D = \{s_i, y_i\}_{i=1}^N$) (Section 4.1) minimizing the cross-entropy loss (L) [35] to support **DG1**.

$$L = -\frac{1}{N} \sum_{i=1}^N \log(P_\theta(y_i | s_i)) \quad (3)$$

3.3. Context Retrieval

Identified NLACPs must be translated into policies aligned with the organizational context to be enforceable. By organizational context, in this research, we refer to the organization-specific information, like subjects, actions, and resources, that should be addressed by the generated policies. However, to achieve that, existing systems require repetitive retraining on the organization’s confidential access requirements whenever the organizational structure changes [5]. Hence, to satisfy **DG2** without costly retraining, CHAGent uses organization-specific information as non-parametric memory when translating NLACPs [36].

CHAGent first generates embedding vectors for organization-specific subjects/roles, actions, and resources, and stores them in vector databases [37]–[39]. At inference, the NLACP’s (s) embedding is compared against stored vectors via cosine similarity [40], and the $top-k$ most relevant entities ($\mathcal{R}_s$) are retrieved through dense retrieval [36]. Unlike sparse retrieval methods such as BM25, dense retrieval captures semantic properties, making it robust to lexical variations such as synonyms.

3.4. Access Control Policy Generation

Using retrieved context ($\mathcal{R}_s$) CHAGent translates NLACPs (s) into access control policies (p) through access control-specific structured information extraction. This parses each NLACP into a structured intermediate format that captures its ACRs, access decisions, and policy components (subject, action, resource, purpose, and condition). We utilize this intermediate representation rather than generating directly in a language like XACML, as LMs struggle with complex formal syntactic structures, reducing generation accuracy [41]. Moreover, unlike token classification-based techniques in existing systems, this approach simultaneously captures both token-level information and semantics-embedded information, enabling granular policy generation even when an NLACP contains multiple ACRs with differing decisions. For example, given “*The doctor can write reports, but the nurse cannot.*”, CHAGent generates the access control policy as $[\{\text{'decision': 'allow', 'subject': 'doctor', 'action': 'write', 'resource': 'report', 'purpose': 'none', 'condition': 'none'}\}, \{\text{'decision': 'deny', 'subject': 'nurse', 'action': 'write', 'resource': 'report', 'purpose': 'none', 'condition': 'none'}\}]$. When applying it to the authorization system, this can be transformed into any language, like XACML, for example, using templates, a popular technique for formulating enforceable policies in industry-grade systems like WSO2 Identity Server [42]. We use this transformation technique in Section 6 to generate RBAC policies from NL.

We employ decoder-only LMs for policy generation, as they excel at structured information extraction and tasks combining natural and domain-specific language after domain adaptation [23], [43]. Specifically, we adopt LLaMA 3 8B, a state-of-the-art open-source LM that achieves superior performance on benchmarks such as HumanEval [44], [45].

Its small footprint enables local deployment even on low-resource hardware, supporting **DG1**, while improving the policy generation performance [22].

In this research, we use the “*instruct*” version of LLaMa 3 8B (P_ϕ) [45]. To support **DG2**, we further fine-tune it using the access control policy dataset, D , containing N input prompts q_i (e.g., $s_i + \mathcal{R}_{s_i}$) with their corresponding access control policies, p_i (Section 4.1) through Parameter Efficient Fine Tuning (PEFT) with low-rank adapters (LoRA) [46]. This helps the LM to be specifically adapted to the access control policy generation context, which is lacking in existing systems. Furthermore, LoRA mitigates catastrophic forgetting during LM fine-tuning by adding a small set of trainable adapter parameters ($\Delta\phi$) and updating only them while keeping the rest of the pre-trained parameters frozen [46]. In the fine-tuning process, we minimize the negative log-likelihood (L) of the generated access control policy.

$$L = -\frac{1}{N} \sum_{i=1}^N \sum_{t=1}^{\tau} \log(P_{\phi+\Delta\phi}(p_{i,t}|q_i, p_{i,<t})) \quad (4)$$

Here, τ is the number of output tokens like “*decision*”. After fine-tuning, we greedily generate the policy ($\hat{y}$), removing the randomness from policy generation [47].

$$\hat{p} \approx \operatorname{argmax}_y P_{\phi+\Delta\phi}(p|q) \text{ where } q = s + \mathcal{R}_s \quad (5)$$

3.4.1. Post-processing. CHAGent post-processes the generated policy by replacing policy components with the most similar components from vector databases (Section 3.3), ensuring organizational-context alignment, supporting **DG2**.

3.5. Access Control Policy Validation & Refinement

Even with access control and organizational context adaptation, LMs may still generate incorrect policies due to hallucinations [48] or NLACP complexity and ambiguity [5], [10]. Validation is therefore essential to mitigate such occasions and improve the policy accuracy [33], [47]. To this end, we introduce an automatic policy validator (**DG3**) that identifies incorrect policies based on the joint representation of each NLACP and its generated policy.

We develop our validator (P_ψ) as a *multi-class classifier*. Unlike binary validators in prior ML research [47], it identifies both correct and incorrect policies along with their error reasons, enhancing system transparency. It outputs a probability distribution over the $c = 12$ classes given the joint representation of the NLACP (s), and the access control policy (p). Given an NLACP (s) and its generated policy (p), CHAGent first reconstructs p as a natural language sentence (r) using a predefined template ($r = h(p)$). This reconstruction allows the validator to more easily detect semantic discrepancies between the NLACP and the generated policy, as NL comparison is found to be more effective in LMs than comparing against a code-like structured format [33], [48]. The concatenation of r and s , is then passed to the validator, which classifies the policy as correct or identifies one of 11 error types: incorrect decision, incorrect subject,

missing subject, incorrect resource, missing resource, incorrect purpose, missing purpose, incorrect condition, missing condition, incorrect action, or missing ACRs.

We select encoder-only or encoder-decoder LMs as the candidates for the base model of our validator. This is due to their ability to incorporate the past and future context of the input in classification [49]. After empirically evaluating CHAGent with encoder-only and encoder-decoder LMs (i.e., BERT [35], BART [50], and T5 [51]) as validators (Appendix B), we use BART LM (g_ψ) [48], [50] as the base architecture for our validator.

We first augmented g_ψ by adding an FFN with two linear layers ($\mathbf{w}_1, \mathbf{b}_1$ and $\mathbf{w}_2, \mathbf{b}_2$) on top of the final layer hidden states of the last decoder token, $[eos]$ of BART [48], to output the classification results.

$$\mathbf{h}_{out} = g_\psi(s, r) \text{ and } \mathbf{z} = \mathbf{w}_2^T (\mathbf{w}_1^T \mathbf{h}_{out} + \mathbf{b}_1) + \mathbf{b}_2 \quad (6)$$

$$P_\psi(c|s, p) = \text{softmax}(\mathbf{z})_c = \frac{e^{z_c}}{\sum_{k=0}^{11} e^{z_k}}, c \in \{0, 1, \dots, 11\} \quad (7)$$

Then, we train it using the access control policy validation dataset, $D = \{s_i, r_i, c_i\}_{i=1}^N$ (Section 4.2) by minimizing the cross-entropy loss (L) between the classifier output and the ground truth label [48], further supporting **DG2**.

$$L = -\frac{1}{N} \sum_{i=1}^N \log(P_\psi(c_i|s_i, r_i)) \quad (8)$$

After the validator identifies the error category, it will be converted into feedback (Appendix D) that instructs the policy generation module to correct the error and re-output the refined policy. To support **DG3**, CHAGent repeats this automatic validation-refinement process at most n times until the correct policy for the given NLACP is generated. If the validation result still indicates the refined policy is incorrect even after n iterations, a ‘‘refinement feedback’’ will be provided to the policy implementers, allowing them to accurately refine it manually, thereby achieving **DG4**.

3.6. Error Localization & Feedback Generation

Reporting only the error type may be insufficient for a human to accurately locate erroneous ACRs, we develop a novel error locator (P_μ) that identifies the start (u) and end (v) positions of erroneous ACRs within a policy, given the NLACP (s), generated policy (p), and error type (c). This is achieved using BERT with two appended linear layers ($\mathbf{w}_{start}, \mathbf{b}_{start}$ and $\mathbf{w}_{end}, \mathbf{b}_{end}$) over the final hidden states (h_k) to score each token’s likelihood (z_k) of being the start or end of an erroneous ACR.

$$z_k^{start} = \mathbf{w}_{start}^T h_k + \mathbf{b}_{start} \text{ and } z_k^{end} = \mathbf{w}_{end}^T h_k + \mathbf{b}_{end} \quad (9)$$

If l is the input token length and $o \in \{u, v\}$,

$$P_\mu(o|s + c, p) = \text{softmax}(z_o) = \frac{e^{z_o}}{\sum_{k=1}^l e^{z_k}} \quad (10)$$

We train the locator with the policy validation dataset improved by adding starting and ending positions of the

TABLE 2: Access control policy dataset. Domains: $\diamond$ conference management, $\star$ healthcare, and $\dagger$ education.

Dataset	non-NLACP	NLACP	ACRs
<i>Cleaned document-folds (1523 sentences) [14]</i>			
Collected ACP (CACP)	28	112	209
CyberChair (CC) $\diamond$	106	104	157
IBM (IBM) $\dagger$	86	114	158
iTrust T2P (T2P) $\star$	48	341	594
iTrust ACRE (ACRE) $\star$	151	433	863
Augmented (Step 1)	522	408	687
LLM-based (Step 2)	0	603	1489
<i>Overall (3056 sentences)</i>			
- Train	749	1848	3677
- Test	192	267	481

erroneous ACRs within each p ($D = \{s_i, p_i, c_i, u_i, v_i\}_{i=1}^N$), by minimizing the following joint cross entropy loss, L_μ .

$$L_\mu = -\frac{1}{N} \sum_{i=1}^N [\log P_\mu(u_i|s_i + c_i, p_i) + \log P_\mu(v_i|s_i + c_i, p_i)] \quad (11)$$

During inference, locator predicts the starting and ending tokens of the erroneous ACRs, $\hat{u}$, and $\hat{v}$.

$$\hat{u} = \text{argmax}_{k=1, \dots, l} P_\mu^{start}(k | s + c, p) \quad (12)$$

$$\hat{v} = \text{argmax}_{k=1, \dots, l} P_\mu^{end}(k | s + c, p) \quad (13)$$

$\hat{u}$ and $\hat{v}$ can be used to derive erroneous ACRs $\hat{e}$.

$$\hat{e} = p[\hat{u} : \hat{v} + 1] \quad (14)$$

Even when an erroneous policy, its error type, and location are provided, unclear communication can still lead to incorrect refinements ([52]). To address this, we generate feedback grounded in [53]’s six Ws of explainable security. The feedback targets policy implementers (**Who**) and provides step-by-step instructions in plain English for resolving policy issues (**How**), alongside the error type and location as shown in Figure 5d (**What**). The feedback is displayed only upon automatic refinement failure (**When**), through either a graphical interface close to the incorrect policy (Section 6.2) [54] or the terminal used to run CHAGent (**Where**).

4. Datasets

Here we present the datasets used to train CHAGent.

4.1. Access Control Policy Dataset

Advances in NLP are hindered in the access control domain by a lack of domain-specific datasets, largely due to the confidential nature of organizational access requirements [7], [11]. To address this, we developed a new dataset from the *only* public real-world requirement specification datasets introduced by Xiao et al. [18], and Slankas et al. [14], covering three systems: iTrust, IBM course registration, and CyberChair. The original annotations include sentence type labels (NLACP or non-NLACP) and entity

annotations (subject, action, resource) for each ACR, but omit access decisions, purposes, and conditions [7], [55]. We cleaned the datasets through deduplication and enriched annotations with access decisions, purposes, and conditions following prior work [55], [56]. To minimize data scarcity and improve LM adaptation, we applied two data generation techniques: data augmentation [7] and LLM-based synthetic data generation [57], [58] (see Appendix C).

Table 2 summarizes the resulting dataset. The *Cleaned document-folds* represent the cleaned real-world documents [14], used exclusively for evaluation following prior work [4], [7], ensuring CHAGent is always evaluated on real-world data even when trained with synthetic data. The “Overall” dataset combines all folds with synthetic data.

4.2. Access Control Policy Validation Dataset

The access control policy validation (ACP) dataset is used to train and evaluate our validator (Section 3.5). Therefore, it should contain sequence pairs representing NLACP and its correct access control policy (i.e., negative pairs) and pairs containing NLACP and its incorrect access control policy (i.e., positive pairs). Using negative pairs from the ACP dataset, we generated positive pairs by applying four manipulation techniques: (a) **Decision change**, randomly altering an ACR’s decision; (b) **Policy component replacement**, substituting a component value with a different one; (c) **Policy component deletion**, replacing a component value with “none”; and (d) **ACR deletion**, removing an ACR. This yielded approximately 2500 incorrect policies per error type. Finally, we assigned labels 0 to 10, depending on the error type for the positive sequence pairs and label 11 for the negative sequence pairs.

4.3. Access Control Policy Refinement Dataset

The access control policy refinement dataset is used to teach the policy generation module to refine an incorrect policy based on the feedback of the validator. Therefore, each training example (Appendix D) of this dataset should contain the NLACP, its incorrectly generated policy, the error type, and the correct policy of the NLACP as the label. All the mentioned components of a training example can be derived from the previously described datasets (i.e., NLACP and its correct policy from the ACP dataset, and the incorrect policy with the error type from the ACP validation dataset).

5. Evaluation and Main Results

We evaluate CHAGent using the F1 score on the document folds (Section 4), following prior research [7]. To determine the number of refinement steps (n), we varied n from 0 to 10 on the “Overall” dataset. As shown in Figure 2, the number of correct policies peaks and stabilizes at $n = 3$, which we adopt for all evaluations. Similarly, we varied k on the same dataset, finding that the highest performance is achieved when $k = 3$. We trained and evaluated components of CHAGent across three random seeds and report the mean F1 score and standard deviation (SD) (i.e., $\text{mean} \pm \text{SD}\%$).

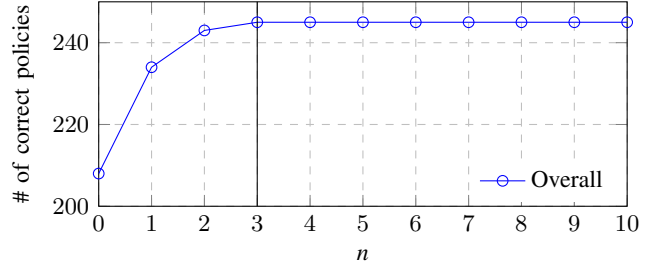


Figure 2: Number of refinement iterations (n) selection.

5.1. Experiment Results

5.1.1. NLACP Identification. We evaluate the performance of CHAGent in identifying NLACPs using both document folds and the “Overall” datasets. According to Table 3, CHAGent achieves an average F1 score across all the document folds of 90.6% and attains the F1 score of 94.8% on the test set of the “Overall” dataset.

5.1.2. Access Control Policy Generation. Given an NLACP, we evaluate CHAGent on access control policy component extraction and access control rule generation. Policy component extraction focuses on extracting individual components for each action in an NLACP [4], [5], [7], [10], [14], [15], while rule generation focuses on translating NLACPs into their intermediate structured representations.

However, direct comparison of policy component extraction performance with prior work is not feasible, as our document folds use corrected entity annotations from Slankas et al. (Section 4). To ensure a fair comparison, we reimplemented the ACR extraction modules from the two most accurate systems that provide sufficient implementation details [5], [7]¹. We reevaluated them with our datasets following the same protocol used to evaluate CHAGent.

The results for access control policy component extraction are reported in Table 4 under two settings of CHAGent: SAR and DSARCP. In the SAR setting, we only consider subjects (S) and resources (R) for a given action (A) to calculate the F1 score, enabling the comparison with prior research [5], [7]. In the DSARCP setting, we include ACR decisions (D), purposes (P), and conditions (C) in addition to S and R when computing the F1 score for each action of NLACPs. When calculating F1 scores, we defined true positives (TP): components that CHAGent identified that correctly match the ground truth, false positives (FP): incorrectly predicted components, and false negatives (FN): components that CHAGent failed to identify. As shown in Table 4, CHAGent achieves an average document-fold F1 score of 82.9% in the SAR setting, outperforming all the existing policy generation systems by at least 41.3 percentage points. When considering access decisions and complex components such as purposes and conditions [55] of each ACR, CHAGent surpasses the existing systems in

1. We contacted the authors for their original implementations but received no response.

TABLE 3: The NLACP identification performance of CHAGent. N/R: results were not reported in the original paper.

	T2P	ACRE	IBM	CACP	CC	Average	Overall
ACRE [14], [15], [29]	N/R	N/R	N/R	N/R	N/R	72%	-
Narouei et al. [26]	74.8%	83.4%	84.4%	72.0%	73.1%	77.5%	-
Xia et al. [7]	95.7%	82.2%	77.5%	89.8%	78.5%	84.7%	-
CHAGent	96.6$\pm$0.9%	92.3$\pm$0.3%	86.4$\pm$0.9%	93.0$\pm$1.2%	80.4$\pm$1.5%	90.6$\pm$0.5%	94.8$\pm$2.6%

TABLE 4: The access control policy component extraction performance of CHAGent. $\diamond$ Extracts subjects, actions, and resources. $\star$ Extracts access decisions, subjects, actions, resources, purposes, and conditions. $\dagger$ With iterative refinement.

	T2P	ACRE	IBM	CACP	CC	Average	Overall
Narouei et al. [4], [5], [26] $\diamond$	12.4%	16.3%	16.8%	26.6%	12.6%	16.9%	-
Xia et al. [7] $\diamond$	48.2%	34.8%	44.7%	50.9%	29.2%	41.6%	-
CHAGent (SAR)$\diamond$	93.5%	73.8%	87.1%	88.92%	71.2%	82.9%	-
CHAGent (DSARCP)$\star\dagger$	93.1$\pm$0.1%	73.0$\pm$1.6%	85.3$\pm$4.2%	88.1$\pm$0.9%	70.0$\pm$0.9%	81.9$\pm$0.9%	83.9$\pm$1.0%

TABLE 5: The ACR generation performance of CHAGent in terms of F1 score. Baselines use few-shot prompting.

	T2P	ACRE	IBM	CACP	CC	Average	Overall
LLaMa 3	55.2%	42.6%	53.3%	54.5%	36.3%	48.4%	-
GPT-5	70.4%	62.2%	79.0%	68.8%	53.2%	66.7%	-
CHAGent	92.8$\pm$0.8%	68.8$\pm$1.9%	82.2$\pm$2.8%	84.0$\pm$2.4%	68.2$\pm$2.0%	79.2$\pm$1.2%	81.4$\pm$0.5%

TABLE 6: The performance of the validator and the error locator.

Metric	Correct	Incorrect						Missing				
		D	S	A	R	P	C	ACR	S	R	P	C
F1 _{validator}		97.7%	97.7%	98.7%	98.3%	100%	99.7%	90.7%	99.7%	100%	95%	92.3%
	88.0 $\pm$ 2.0%	99.0 $\pm$ 0.0%										
F1 _{validator-macro}	93.7 $\pm$ 1.5%											
F1 _{error locator}	96.3%											

the DSARCP setting, yielding an average F1 score of 81.9%. It achieves the F1 score of 83.9% on the “Overall” dataset.

Evaluating policy generation on individual components does not reflect real-world performance: if a system identifies the subject/role but misses the resource, according to the evaluation followed by prior work [4], [5], [7], the F1 score is 66.7%, whereas the correct score should be 0 since the ACR is incomplete. Instead, we evaluate CHAGent on complete ACR generation. As shown in Table 5, CHAGent achieves an average F1 of 79.2% across document folds and 81.4% on the “Overall” dataset. Once evaluated under the same evaluation protocol, it outperforms the generic/untrained LLaMa 3 as well as much larger closed-source LLMs like GPT-5 with few-shot prompting.

5.1.3. Access Control Policy Validation and Error Localization. To evaluate the CHAGent’s validator, we utilized the policy validation dataset based on “Overall” dataset shown in Table 2, as it contains different types of NLACPs from different domains (e.g., healthcare, education, etc.). We used its train (80%), test (10%), and validation sets (10%) randomly, to train the validator and evaluate its performance, respectively. The obtained evaluation results (Table 6) indicate that CHAGent’s validator identifies incorrect and correct access control policies with F1 scores of 99.0% and

88.0%, respectively, achieving a macro F1 score of 93.7%. Also, the error locator identifies the erroneous ACRs of a generated policy with a macro F1 score of 96.3%.

To further analyze the validator’s errors, we examined the confusion matrix for the best-performing seed, shown in Appendix E. It shows that while the validator correctly identified 2791/2813 incorrect policies, it resulted in 18 FPs (0.59%) and 22 (0.73%) FNs. Most frequent misclassifications occur between missing ACRs and correct policies. Moreover, 35 of the classification errors classified the policy as erroneous but assigned it to a different error category. This is mainly due to the confusion between missing conditions and purposes (9).

5.2. Ablation Study

The following ablation study highlights the sources of CHAGent’s significant performance improvement.

5.2.1. Effect of Utilizing Contextual Information. As shown in Figure 3, omitting organizational context from policy generation (i.e., using the fine-tuned model alone for generation) causes substantial F1 score drops across all document folds, with reductions of 24.6%, 21.3%, and 9.9% for CC, CACP, and IBM, respectively. The drops are smaller

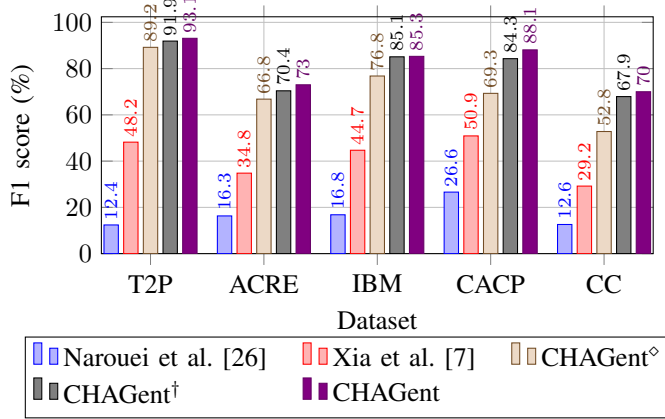


Figure 3: Effect of organizational context. [◇] w/o context + refinement, [†] w/o post-processing + refinement.

for T2P (4.2%) and ACRE (8.4%), as both are derived from the same iTrust dataset [14] and share similar entities. Consequently, the model partially learns these entities during training, reducing reliance on organizational context.

Figure 3 also shows that omitting post-processing (Section 3.4.1) slightly reduces performance, indicating that providing contextual information at both generation and post-processing stages yields the best results.

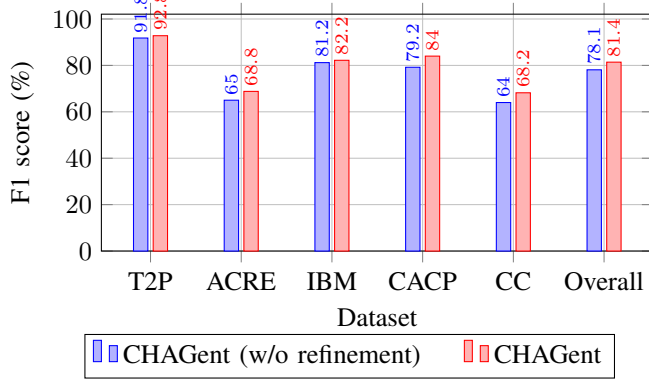


Figure 4: Effect of iteratively refining the generated policies.

5.2.2. Effect of Iterative Refinement. The effect of the iterative refinement on the policy generation performance is shown in Figure 4. CHAGent achieves the highest ACR generation performance improvement of 6.6% for CC. That is because, compared to the other folds, CC contains many ambiguous NLACPs (e.g., NLACPs without subjects), making it harder for CHAGent to generate policies accurately at once (due to hallucinations), requiring further refinements. It also gains a 4.2% improvement for “Overall” dataset.

6. User Study

We conducted think-aloud studies with professional policy implementers to empirically assess CHAGent’s effec-

tiveness. Participants performed policy generation tasks using a system built on CHAGent while verbalizing their thoughts, followed by post-study semi-structured interviews. To our knowledge, this is the first empirical evaluation of an automated access control policy generation system with professional policy implementers. The study was approved by the ethics committee of our university (Appendix A).

6.1. Participant Recruitment

We recruited 17 policy implementers via Prolific [59] using a screening survey targeting full-time professionals with access control policy generation experience. Participants included system administrators (P4, P8, P11), IT officers (P2, P6, P14), cybersecurity specialists (P7, P9, P10, P12, P17), DevOps engineers (P1, P3), and upper management roles such as CTOs (P5, P13, P15, P16), all with RBAC implementation experience across different domains. More details about the participants can be found in Appendix F. We provided the participants with approximately 24 USD.

6.2. User Interface (UI)

We designed the UI to allow participants to easily interact with CHAGent’s implementation and generate RBAC policies (since all the participants are familiar with RBAC). We refer to this entire system (i.e., CHAGent + UI) as CHAGentSys. This study focuses solely on participants’ feedback about CHAGentSys’s performance; UI usability is left for future work.

CHAGentSys’s UI supports two modes: uploading a full requirement specification document or entering an individual NLACP (Figures 5b, 5c). Before generation, users upload an organization hierarchy that enables organizational context retrieval (Section 3.3). When an incorrect policy is generated, the refinement page (Figure 5d) displays the original NLACP, the generated policy as a table of ACRs and components, and refinement feedback (Section 3.6). Users can then make modifications to correct policies by selecting components from drop-down menus populated with organization hierarchy values, minimizing human errors from free-form input [60]. The UI also supports policy testing (Figure 5e) by transforming generated policies into XACML and storing them in a policy database. Users can build access requests by selecting policy components and submit them through a built-in policy decision point (PDP), which applies first-applicable rule combining and deny-overrides policy combining algorithms, returning either *allow* or *deny*.

CHAGentSys was deployed on a server with one Nvidia A100 80GB GPU, utilizing approximately half its memory (≈ 40 GB) due to its SLM-based implementation.

6.3. Think-aloud Study

We first ran two pilot studies with P01 and P02 (Table 8) and found that the number of policies they had to generate was excessive, leaving participants fatigued before the interview, reducing the quality of their answers. After refining

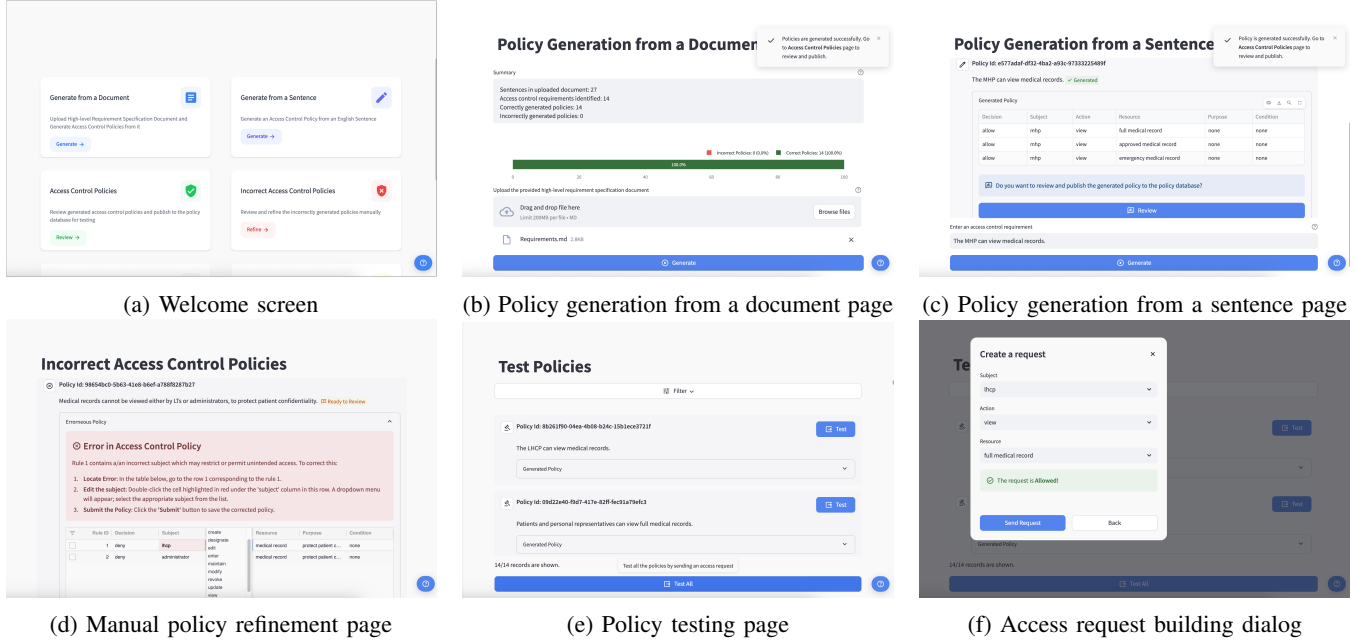


Figure 5: Pages of CHAGentSys

the study material, we ran 17 think-aloud sessions with participants (P1-P17) where they performed policy generation tasks using CHAGentSys via Zoom. We recorded audio and participants’ computer screens for analysis. At each session, the participant was provided a high-level requirement specification document (inspired by the access requirements from the iTrust electronic health record system [14], [61]) and the organizational hierarchy. Participants completed two tasks while verbalizing their reasoning. In Task 1 (10 minutes), participants translated the specified access requirements into RBAC policies using CHAGentSys. Task 2 (15 minutes) consisted of a questionnaire comprising yes/no verification questions about the policies generated in Task 1, as well as open-ended policy generation questions targeting a specified goal. Unlike Task 1, Task 2 provided no pre-written requirements; participants were required to independently author and translate their own access requirements, affording an opportunity to explore CHAGentSys’s capabilities and identify its strengths and limitations. These tasks required them to generate simple policies as well as complex cases like policies containing permission inheritance and conflicts. Each session concluded with a semi-structured interview.

6.4. Data Analysis

Collected data were analyzed using inductive reflexive thematic analysis [62], chosen for its flexibility and capacity to incorporate researcher domain knowledge in access control and machine learning [62]. The lead researcher transcribed and repeatedly reviewed the recordings to familiarize themselves with the data, then applied semantic coding [63] to produce an initial codebook and derive themes reflecting participants’ perceptions of CHAGentSys and its features,

including the verification and feedback mechanisms. Co-authors subsequently reviewed, refined, and extended the codes and themes through collaborative reflection.

6.5. User Study Results

Across both tasks, CHAGentSys correctly translated 256 of 276 access requirements (92.75%). Of the 20 failed cases, 19 were detected and flagged for manual refinement by CHAGentSys’s validator, but one from P16 remained undetected. Mean task completion times were 9.8 minutes for Task 1 and 14.6 minutes for Task 2.

6.5.1. CHAGentSys accurately translates complex access requirements. CHAGentSys correctly translated 204/221 NLACPs during Task 1 and generated correct policies for all NLACPs provided by participants P2–P12, P14, P15, and P17 during Task 2. These were independently defined and written by participants in their own styles, unlike in Task 1. Correctness was validated both syntactically and semantically by CHAGentSys’s validation and by participants through policy testing. Consequently, all participants perceived CHAGentSys as highly accurate, with several noting that it interpreted NLACPs more accurately than they could themselves.

P9: “...in some cases the system understood [the access requirement] better than me”

According to P9, this is particularly important because access requirements are often written by another person (i.e., a policy maker), leading policy implementers to misinterpret them when manually generating policies.

P9: “...due to the previous work being manual, it was susceptible to introducing errors. Maybe

I could have misunderstood the intention of the requirement, and I could apply a different policy.”

The participants also highlighted CHAGentSys’s ability to translate complex access requirements that affect multiple roles and resources accurately.

P11: *“complex policy applying for different users ...I can see that would be simplified”*

Nonetheless, during Task 2, CHAGentSys generated an incorrect policy and was also missed by its validator due to its ambiguity. Upon resolving the ambiguity by P16, CHAGentSys correctly translated the input NLACP.

6.5.2. CHAGentSys’s performance motivates its adoption. Participants consistently cited CHAGentSys’s strong performance as a key motivator for adoption, noting its potential to mitigate human error in large-scale manual policy generation and expressing willingness to deploy it within their organizations.

P2: *“It will ...surpass the issue of [human] mistakes ...so it’s a very good thing that if it can be implemented in an organization like ours ...because we also do have a lot of employees with us.”*

Similarly, P5 and P16 perceived CHAGentSys as more accurate than ChatGPT (i.e., GPT-4), indicating their preference to replace existing ChatGPT-based policy generation approaches in their organizations

P16: *“... instead of using ChatGPT, I will easily do it [use CHAGentSys], because ChatGPT, I will need to request it several times to have it right [prompt ChatGPT repetitively to generate a policy correctly], [But] here once, and it got it right.”*

Finally, participants further highlighted that CHAGentSys’s organizational context-awareness gained using organizational hierarchy is a key requirement for adopting CHAGentSys to automate manual policy generation.

P10: *“If LM is working and aware of the situation in the ...organizational structure ...there’s not really a situation where you’d want to go back and add them [policies] manually.”*

6.5.3. CHAGentSys’s validation and explainable feedback aid accurate policy refinement. When CHAGentSys generated an incorrect policy, its validation mechanism identified the error and provided explainable feedback, enabling 16 participants to correctly make refinements.

P17: *“[The refinement feedback was] very straightforward, very understandable. The errors were clear; ...the way that you guided me to fix them was excellent.”*

However, P1 and P8 found the refinement feedback overwhelming, struggling to refine the incorrect policy in Task 1. While P1 ultimately succeeded, P8 failed to refine the policy accurately.

P1: *“[Feedback is] pretty understandable ...I feel it might have explained a little too much.”*

6.5.4. CHAGentSys’s automated validation and local deployability establish trust. Trust is critical for adopting AI systems like CHAGentSys in real-world organizations [64]. As P2 noted, CHAGentSys’s local deployability established initial trust by eliminating privacy concerns associated with sending sensitive access requirements to third-party services.

P2: *“Well, I do trust it. 100%. ...because, well, [it is] just a stand-alone program. It’s not connected to any [third-party] AI piece that we are not sure of, so I do trust it. 100% ...especially when you are dealing with sensitive healthcare information.”*

Participants’ trust increased as they observed CHAGentSys’s accurate translations, with P9 suggesting they could “rely on the results, reviewing only a sample [of policies]” in a large-scale policy generation. Also CHAGentSys’s validation mechanism improves the transparency of the policy generation process, further strengthening user trust.

P10: *“I think it improves trust, because obviously, you can see that the system is reviewing the inputs and then provides guidance or tells the end user when there’s something wrong there.”*

7. Discussion

We presented CHAGent, a novel human-centric, context-aware access control policy generation framework, introducing a new policy validation, refinement, error localization, and feedback loop for accurate policy generation. We demonstrated that CHAGent substantially improves policy generation accuracy compared to prior approaches. Its local deployment capability preserves the confidentiality/privacy of access requirements, fostering user trust and encouraging adoption [64], demonstrating the importance of **DG1**. This suggests that privacy-preserving deployment is not merely a practical consideration but a fundamental requirement for the real-world applicability of AI-based security tools.

A central finding of this work is the importance of context awareness (**DG2**) in access control policy generation. Incorporating organizational context enables the policy generation systems to align generated policies with existing roles, actions, and resources, reducing inconsistencies that often occur in policy generation [5], [6], [65]. This is particularly critical in real-world environments, where organizational structures are dynamic and domain-specific [3]. Our evaluation shows that removing contextual information leads to significant performance degradation, highlighting that policy generation cannot be treated as a purely language understanding task, as in prior work [5], [7], [25]). Instead, it must be grounded in organization-specific knowledge. Beyond improving automated policy generation, CHAGent’s organizational context awareness also helps mitigate human errors in manual policy generation. As P10 highlighted, human errors can occur when they generate policies that “inherit something from a specific team leader. But then there will be a few branches where specific users cannot do a specific thing.” This is because, to generate such policies and manage exceptions, policy implementers should have

an up-to-date knowledge of the organization’s structure, which is harder to retain [3]. However, incorporating organization structure when generating policies, allows policy implementers to handle such complex situations accurately, reducing their cognitive burden.

The results also demonstrate the effectiveness of domain adaptation (**DG2**) in improving policy generation. Even without organization-specific entities, CHAGent outperforms existing systems (Section 5.2.1), owing to its access control policy context adaptation (**DG2**), which enables it to learn nuances that generic models and even humans sometimes lack [3]. User study participants noted that CHAGent outperforms more powerful LLMs used in their organizations and, in some cases, interprets access requirements better than they could themselves. Such perceptions foster trust and drive adoption [64], [66]. This suggests that domain-specific fine-tuning (that prior systems lack [4], [6], [7]) remains a critical factor for applying AI in security-sensitive tasks, where generic models often fail to capture subtle but important distinctions [5], [6].

Another important contribution is the introduction of an automated validation and refinement mechanism (**DG3**). Given the inherent fallibility of ML models [1], [4], [6], [67], validation plays a crucial role in identifying incorrect policy generations and improving overall system reliability. Our results show that iterative refinement guided by this validation leads to significant performance gains, particularly for complex or ambiguous requirements.

Nonetheless, the validator produces a small number of FPs that misidentify correct access control policies as incorrect, often because the generated policy is missing an ACR. This is primarily due to the complexity and ambiguity of multi-rule policies, where the boundary between a complete policy and one with a subtle omission is difficult to distinguish. Consequently, the F1 score for identifying missing ACR becomes relatively lower due to low precision. Similarly, the confusion between the missing condition and purpose suggests that these contextual components can be difficult to distinguish, especially when their meanings overlap in the NLACP. Thus, even when the validator correctly detects that a policy is erroneous, it may assign the wrong error category. However, these errors are less critical because they still trigger refinement and typically allow the generation module or policy implementer to correct the policy. In contrast, FNs, incorrect policies passing validation (0.73%), represent a potential security risk. To mitigate this, CHAGentSys adopts test-driven human-in-the-loop verification, ensuring that CHAGentSys remains a safe assistive system rather than a fully autonomous decision-maker.

The human-in-the-loop paradigm plays a key role in the CHAGent’s design. Rather than fully automating policy generation, CHAGent supports users by automating repetitive tasks while providing mechanisms to identify and correct errors that cannot be fixed automatically, through explainable feedback (**DG4**). Feedback improved self-efficacy in correctly refining incorrect policies, thereby avoiding trial-and-error [52]. Consequently, participants successfully refined incorrect generations, improving overall policy accuracy

by 6.5%. This highlights the importance of explainability in security-critical systems. At the same time, feedback presentation must be carefully balanced, as overly detailed feedback may overwhelm users. Future systems should consider adaptive or progressive feedback mechanisms to better support users at different levels of expertise.

CHAGent’s automated validation (**DG3**) and explainable feedback improved both policy generation accuracy and participants’ confidence in CHAGentSys’s outputs by enhancing transparency. Allowing users to address errors gave them a sense of ownership of the generated policies, strengthening their trust in CHAGent. This aligns with prior research, suggesting that the explanations given by AI systems on their failures play a key role in establishing trust in the system, especially for novice users [68].

Overall, CHAGent shows that reliable access control policy generation requires more than language understanding. It requires domain adaptation, organizational grounding, validation, and targeted human oversight. These findings provide a broader design direction for building AI-assisted security tools: automation should reduce manual effort while preserving human control over security-critical decisions.

8. Limitations and Future Work

This study solely focuses on accurately translating high-level requirements into access control policies. However, the generated policies must also be consistent, correct, minimal, relevant, and complete [21]. While we ensure policy correctness through automated validation, CHAGent can be extended, as future work, using techniques such as Binary Decision Diagrams [69], [70] to enforce consistency, minimality, relevance, and completeness.

9. Conclusion

We presented CHAGent, a novel human-centric context-aware access control policy generation framework. It is designed to help policy implementers accurately generate access control policies from high-level requirement specifications. CHAGent can be implemented using locally deployable SLMs without compromising policy generation performance, thereby ensuring confidentiality throughout the process. CHAGent translates complex access requirements by leveraging specialized knowledge gained through adaptation to the access control policy and organizational context. To address the inherent fallibility of automated policy generation, CHAGent introduces a validation and refinement loop that detects incorrect policies, attempts automatic correction, and, when needed, supports human-in-the-loop refinement through error localization and explainable feedback. Our evaluation showed that CHAGent improves policy generation accuracy over prior approaches. The user study with professional policy implementers demonstrated that its context awareness, validation, and feedback mechanisms improve accuracy, trust, and willingness to adopt the system. We also introduced three datasets, addressing the data scarcity in the domain.

LLM Usage Statement

LLMs were used for editorial purposes in this manuscript, and all outputs were inspected by the authors to ensure accuracy and originality. Language models are also part of the research methodology described in this paper, including policy generation, validation, refinement, error localization, synthetic data generation, and baseline comparison. Their use is explicitly reported in the relevant methodology and evaluation sections. The authors validated the outputs through the experiments and user study reported in the paper and take full responsibility for the correctness, originality, and integrity of the submitted work.

References

- [1] S. H. Jayasundara, N. A. G. Arachchilage, and G. Russello, "Vision: 'accessformer': Feedback-driven access control policy generation framework," *Symposium on Usable Security and Privacy (USEC)*, 2024.
- [2] S. A. Horstmann, S. Domiks, M. Gutfleisch, M. Tran, Y. Acar, V. Moonsamy, and A. Naiakshina, "Those things are written by lawyers, and programmers are reading that. mapping the communication gap between software developers and privacy experts," *Proceedings on Privacy Enhancing Technologies*, 2024.
- [3] L. Bauer, L. F. Cranor, R. W. Reeder, M. K. Reiter, and K. Vaniea, "Real life challenges in access-control management," in *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, 2009, pp. 899–908.
- [4] M. Narouei and H. Takabi, "Automatic top-down role engineering framework using natural language processing techniques," in *IFIP International Conference on Information Security Theory and Practice*. Springer, 2015, pp. 137–152.
- [5] M. Narouei, H. Takabi, and R. Nielsen, "Automatic extraction of access control policies from natural language documents," *IEEE Transactions on Dependable and Secure Computing*, vol. 17, no. 3, pp. 506–517, 2018.
- [6] S. H. Jayasundara, N. A. Gamagedara Arachchilage, and G. Russello, "Sok: Access control policy generation from high-level natural language requirements," *ACM Computing Surveys*, vol. 57, no. 4, pp. 1–37, 2024.
- [7] Y. Xia, S. Zhai, Q. Wang, H. Hou, Z. Wu, and Q. Shen, "Automated extraction of abac policies from natural-language documents in healthcare systems," in *2022 IEEE International Conference on Bioinformatics and Biomedicine (BIBM)*. IEEE, 2022, pp. 1289–1296.
- [8] M. Narouei and H. Takabi, "Towards an automatic top-down role engineering approach using natural language processing techniques," in *Proceedings of the 20th ACM Symposium on Access Control Models and Technologies*, 2015, pp. 157–160.
- [9] M. Narouei, H. Khanpour, and H. Takabi, "Identification of access control policy sentences from natural language policy documents," in *IFIP Annual Conference on Data and Applications Security and Privacy*. Springer, 2017, pp. 82–100.
- [10] J. Heaps, R. Krishnan, Y. Huang, J. Niu, and R. Sandhu, "Access control policy generation from user stories using machine learning," in *IFIP Annual Conference on Data and Applications Security and Privacy*. Springer, 2021, pp. 171–188.
- [11] M. Alohal, H. Takabi, and E. Blanco, "Automated extraction of attributes from natural language attribute-based access control (abac) policies," *Cybersecurity*, vol. 2, no. 1, pp. 1–25, 2019.
- [12] —, "A deep learning approach for extracting attributes of abac policies," in *Proceedings of the 23rd ACM on Symposium on Access Control Models and Technologies*, 2018, pp. 137–148.
- [13] —, "Towards an automated extraction of abac constraints from natural language policies," in *IFIP International Conference on ICT Systems Security and Privacy Protection*. Springer, 2019, pp. 105–119.
- [14] J. Slankas, X. Xiao, L. Williams, and T. Xie, "Relation extraction for inferring access control rules from natural language artifacts," in *Proceedings of the 30th annual computer security applications conference*, 2014, pp. 366–375.
- [15] J. Slankas and L. Williams, "Access control policy extraction from unconstrained natural language text," in *2013 International Conference on Social Computing*. IEEE, 2013, pp. 435–440.
- [16] —, "Classifying natural language sentences for policy," in *2012 IEEE International Symposium on Policies for Distributed Systems and Networks*. IEEE, 2012, pp. 33–36.
- [17] Y. Zhang, Z. Xu, D. Huo, X. Guo, Q. Zhou, and Y. Wang, "Litroacp: A lightweight and robust framework for extracting access control policies from specifications," in *International Conference on Advanced Information Systems Engineering*. Springer, 2025, pp. 75–90.
- [18] X. Xiao, A. Paradkar, S. Thummalapenta, and T. Xie, "Automated extraction of security policies from natural-language software documents," in *Proceedings of the ACM SIGSOFT 20th International Symposium on the Foundations of Software Engineering*, 2012, pp. 1–11.
- [19] M. Kaur, M. van Eeten, M. Janssen, K. Borgolte, and T. Fiebig, "Human factors in security research: Lessons learned from 2008–2018," *arXiv preprint arXiv:2103.13287*, 2021.
- [20] J. M. Del Alamo, D. S. Guaman, B. García, and A. Diez, "A systematic mapping study on automated analysis of privacy policies," *Computing*, pp. 1–24, 2022.
- [21] E. Bertino, A. A. Jabal, S. Calo, D. Verma, and C. Williams, "The challenge of access control policies quality," *Journal of Data and Information Quality (JDIQ)*, vol. 10, no. 2, pp. 1–6, 2018.
- [22] A. Pareja, N. S. Nayak, H. Wang, K. Killamsetty, S. Sudalairaj, W. Zhao, S. Han, A. Bhandwadar, G. Xu, K. Xu *et al.*, "Unveiling the secret recipe: A guide for supervised fine-tuning small llms," *arXiv preprint arXiv:2412.13337*, 2024.
- [23] J. Dagdelen, A. Dunn, S. Lee, N. Walker, A. S. Rosen, G. Ceder, K. A. Persson, and A. Jain, "Structured information extraction from scientific text with large language models," *Nature Communications*, vol. 15, no. 1, p. 1418, 2024.
- [24] S. Lawal, X. Zhao, A. Rios, R. Krishnan, and D. Ferraiolo, "Translating natural language specifications into access control policies by leveraging large language models," in *2024 IEEE 6th International Conference on Trust, Privacy and Security in Intelligent Systems, and Applications (TPS-ISA)*. IEEE, 2024, pp. 361–370.
- [25] A. Vatsa, P. Patel, and W. Eiers, "Synthesizing access control policies using large language models," in *2025 IEEE/ACM International Workshop on Natural Language-Based Software Engineering (NLBSE)*. IEEE, 2025, pp. 13–16.
- [26] M. Narouei, H. Khanpour, H. Takabi, N. Parde, and R. Nielsen, "Towards a top-down policy engineering framework for attribute-based access control," in *Proceedings of the 22nd ACM on Symposium on Access Control Models and Technologies*, 2017, pp. 103–114.
- [27] M. N. Nobi, M. Gupta, L. Prahara, M. Abdelsalam, R. Krishnan, and R. Sandhu, "Machine learning in access control: A taxonomy and survey," *arXiv preprint arXiv:2207.01739*, 2022.
- [28] A. Abu Jabal, E. Bertino, J. Lobo, M. Law, A. Russo, S. Calo, and D. Verma, "Polisma-a framework for learning attribute-based access control policies," in *European Symposium on Research in Computer Security*. Springer, 2020, pp. 523–544.

- [29] J. Slankas and L. Williams, "Access control policy identification and extraction from project documentation," *SCIENCE*, vol. 2, no. 3, pp. 145–159, 2013.
- [30] M.-C. De Marneffe, B. MacCartney, C. D. Manning *et al.*, "Generating typed dependency parses from phrase structure parses," in *Lrec*, vol. 6, 2006, pp. 449–454.
- [31] S. Alqurashi, I. Ray, M. Abdelgawad, and H. Shirazi, "Sr2acm: A methodical approach for translating natural language security requirements to access control model," in *2024 IEEE 6th International Conference on Trust, Privacy and Security in Intelligent Systems, and Applications (TPS-ISA)*. IEEE, 2024, pp. 303–312.
- [32] Firstpost, "Ai companies are finally looking at small language models, and expect to make big bucks," May 2024.
- [33] M. Skreta, N. Yoshikawa, S. Arellano-Rubach, Z. Ji, L. B. Kristensen, K. Darvish, A. Aspuru-Guzik, F. Shkurti, and A. Garg, "Errors are useful prompts: Instruction guided task programming with verifier-assisted iterative prompting," *arXiv preprint arXiv:2303.14100*, 2023.
- [34] A. Gasparetto, M. Marcuzzo, A. Zangari, and A. Albarelli, "A survey on text classification algorithms: From text to predictions," *Information*, vol. 13, no. 2, p. 83, 2022.
- [35] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," *arXiv preprint arXiv:1810.04805*, 2018.
- [36] Y. Ding, W. Fan, L. Ning, S. Wang, H. Li, D. Yin, T.-S. Chua, and Q. Li, "A survey on rag meets llms: Towards retrieval-augmented large language models," *arXiv preprint arXiv:2405.06211*, 2024.
- [37] Y. Han, C. Liu, and P. Wang, "A comprehensive survey on vector database: Storage and retrieval technique, challenge," *arXiv preprint arXiv:2310.11703*, 2023.
- [38] N. Muennighoff, N. Tazi, L. Magne, and N. Reimers, "Mteb: Massive text embedding benchmark," *arXiv preprint arXiv:2210.07316*, 2022.
- [39] S. Lee, A. Shakir, D. Koenig, and J. Lipp. (2024) Open source strikes bread - new fluffy embeddings model. [Online]. Available: <https://www.mixedbread.ai/blog/mxbai-embed-large-v1>
- [40] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, and H. Wang, "Retrieval-augmented generation for large language models: A survey," *arXiv preprint arXiv:2312.10997*, 2023.
- [41] M. B. Moumoula, A. K. Kabore, J. Klein, and T. F. Bissyande, "Evaluating programming language confusion," *arXiv preprint arXiv:2503.13620*, 2025.
- [42] WSO2, "Wso2 identity server documentation," Oct 2025. [Online]. Available: <https://is.docs.wso2.com/en/5.9.0/>
- [43] B. Roziere, J. Gehring, F. Gloeckle, S. Sootla, I. Gat, X. E. Tan, Y. Adi, J. Liu, T. Remez, J. Rapin *et al.*, "Code llama: Open foundation models for code," *arXiv preprint arXiv:2308.12950*, 2023.
- [44] M. Chen, J. Twarek, H. Jun, Q. Yuan, H. P. d. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman *et al.*, "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021.
- [45] M. AI, "Introducing meta llama 3: The most capable openly available llm to date," Apr 2024. [Online]. Available: <https://ai.meta.com/blog/meta-llama-3>
- [46] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "Lora: Low-rank adaptation of large language models," *arXiv preprint arXiv:2106.09685*, 2021.
- [47] A. Ni, S. Iyer, D. Radev, V. Stoyanov, W.-t. Yih, S. Wang, and X. V. Lin, "Lever: Learning to verify language-to-code generation with execution," in *International Conference on Machine Learning*. PMLR, 2023, pp. 26 106–26 128.
- [48] J. Shen, Y. Yin, L. Li, L. Shang, X. Jiang, M. Zhang, and Q. Liu, "Generate & rank: A multi-task framework for math word problems," *arXiv preprint arXiv:2109.03034*, 2021.
- [49] A. Patel, B. Li, M. S. Rasooli, N. Constant, C. Raffel, and C. Callison-Burch, "Bidirectional language models are also few-shot learners," *arXiv preprint arXiv:2209.14500*, 2022.
- [50] M. Lewis, Y. Liu, N. Goyal, M. Ghazvininejad, A. Mohamed, O. Levy, V. Stoyanov, and L. Zettlemoyer, "Bart: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension," *arXiv preprint arXiv:1910.13461*, 2019.
- [51] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu, "Exploring the limits of transfer learning with a unified text-to-text transformer," *The Journal of Machine Learning Research*, vol. 21, no. 1, pp. 5485–5551, 2020.
- [52] T. Xu, H. M. Naing, L. Lu, and Y. Zhou, "How do system administrators resolve access-denied issues in the real world?" in *Proceedings of the 2017 CHI Conference on Human Factors in Computing Systems*, 2017, pp. 348–361.
- [53] L. Vigano and D. Magazzeni, "Explainable security," in *2020 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW)*. IEEE, 2020, pp. 293–300.
- [54] J. Nielsen, "Ten usability heuristics," 2005.
- [55] C. A. Brodie, C.-M. Karat, and J. Karat, "An empirical study of natural language parsing of privacy policy rules using the sparcl policy workbook," in *Proceedings of the second symposium on Usable privacy and security*, 2006, pp. 8–19.
- [56] L. Yang, X. Chen, Y. Luo, X. Lan, and L. Chen, "Purext: Automated extraction of the purpose-aware rule from the natural language privacy policy in iot," *Security and Communication Networks*, vol. 2021, 2021.
- [57] M. Schmidhuber and U. Kruschwitz, "Llm-based synthetic datasets: Applications and limitations in toxicity detection," *LREC-COLING 2024*, p. 37, 2024.
- [58] L. Long, R. Wang, R. Xiao, J. Zhao, X. Ding, G. Chen, and H. Wang, "On llms-driven synthetic data generation, curation, and evaluation: A survey," *arXiv preprint arXiv:2406.15126*, 2024.
- [59] B. D. Douglas, P. J. Ewell, and M. Brauer, "Data quality in on-line human-subjects research: Comparisons between mturk, prolific, cloudresearch, qualtrics, and sona," *Plos one*, vol. 18, no. 3, p. e0279720, 2023.
- [60] R. W. Reeder, C.-M. Karat, J. Karat, and C. Brodie, "Usability challenges in security and privacy policy-authoring interfaces," in *IFIP Conference on Human-Computer Interaction*. Springer, 2007, pp. 141–155.
- [61] A. Meneely, B. Smith, and L. Williams, "Appendix b: itrust electronic health care system case study," *Software and Systems Traceability*, p. 425, 2012.
- [62] V. Braun and V. Clarke, "Reflecting on reflexive thematic analysis," *Qualitative research in sport, exercise and health*, vol. 11, no. 4, pp. 589–597, 2019.
- [63] —, "One size fits all? what counts as quality practice in (reflexive) thematic analysis?" *Qualitative research in psychology*, vol. 18, no. 3, pp. 328–352, 2021.
- [64] S. J. Daly, A. Wiewiora, and G. Hearn, "Shifting attitudes and trust in ai: Influences on organizational ai adoption," *Technological Forecasting and Social Change*, vol. 215, p. 124108, 2025.
- [65] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel *et al.*, "Retrieval-augmented generation for knowledge-intensive nlp tasks," *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459–9474, 2020.
- [66] K. A. Hoff and M. Bashir, "Trust in automation: Integrating empirical evidence on factors that influence trust," *Human factors*, vol. 57, no. 3, pp. 407–434, 2015.
- [67] I. F. del Amo, J. A. Erkoyuncu, R. Roy, R. Palmarini, and D. Onoufriou, "A systematic review of augmented reality content-related techniques for knowledge transfer in maintenance applications," *Computers in Industry*, vol. 103, pp. 47–71, 2018.

- [68] S. Lockey, N. Gillespie, D. Holm, and I. A. Someh, “A review of trust in artificial intelligence: Challenges, vulnerabilities and future directions,” 2021.
- [69] A. A. Jabal, M. Davari, E. Bertino, C. Makaya, S. Calo, D. Verma, A. Russo, and C. Williams, “Methods and tools for policy analysis,” *ACM Computing Surveys (CSUR)*, vol. 51, no. 6, pp. 1–35, 2019.
- [70] C. Morisset, T. A. Willemse, and N. Zannone, “A framework for the extended evaluation of abac policies,” *Cybersecurity*, vol. 2, no. 1, p. 6, 2019.
- [71] N. Ng, K. Yee, A. Baevski, M. Ott, M. Auli, and S. Edunov, “Facebook fair’s wmt19 news translation task submission,” in *Proc. of WMT*, 2020.
- [72] M. L. McHugh, “Interrater reliability: the kappa statistic,” *Biochemia medica*, vol. 22, no. 3, pp. 276–282, 2012.
- [73] J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altenschmidt, S. Altman, S. Anadkat *et al.*, “Gpt-4 technical report,” *arXiv preprint arXiv:2303.08774*, 2023.
- [74] AI@Meta, “Llama 3 model card,” 2024. [Online]. Available: https://github.com/meta-llama/llama3/blob/main/MODEL_CARD.md
- [75] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. Chi, Q. V. Le, D. Zhou *et al.*, “Chain-of-thought prompting elicits reasoning in large language models,” *Advances in Neural Information Processing Systems*, vol. 35, pp. 24 824–24 837, 2022.
- [76] S. Kambhampati, K. Valmeekam, L. Guan, K. Stechly, M. Verma, S. Bhamri, L. Saldyt, and A. Murthy, “Llms can’t plan, but can help planning in llm-modulo frameworks,” *arXiv preprint arXiv:2402.01817*, 2024.

Appendix

1. Ethics considerations

We have carefully considered the ethics in accordance with the conference guidelines. Our research aims to enhance the accuracy/performance of the automated access control policy generation process in a human-centric manner. In addition to the theoretical evaluation of our policy generation framework, we also implemented it as a system with a user interface and empirically evaluated it with professional policy implementers. To this end, we received approval from the Human Participants Ethics Committee of the lead author’s university for conducting the user study reported in the paper.

2. Model Selection for the Validator

In this work, we treat the choice of the base model for our access control policy validator as a hyperparameter and use the best-performing model among BERT [35], BART [50], and T5 [51] for further experiments. To measure the performance of each model, we first fine-tune them using the access control policy validation dataset based on the training set of “Overall” dataset (Section 4). Then, we evaluate CHAGent on the test set of the “Overall” dataset three times by replacing the policy validator with fine-tuned BERT, BART, and T5 LMs. The achieved performances of CHAGent each time in terms of the access control rule generation F1 score are shown in Table 7.

According to Table 7, CHAGent performs better when paired with BART in access control rule generation. Hence, in this research, we select BART as the base model for our access control policy validator.

TABLE 7: The access control rule generation performance of CHAGent with different validators.

Base LM	Architecture	F1 Score
BERT	Encoder only	79.2%
T5	Encoder-decoder	80.2%
BART	Encoder-decoder	81.4%

3. Access Control Policy Dataset

As noted in Section 4.1, we improved the original dataset introduced by Slankas *et al.* [14] using data augmentation and LLM-based synthetic data generation. For data augmentation, we used BERT embedding-based word substitution and back translation [7]. For example, BERT embedding-based substitution can transform “*The doctor can read patient’s records.*” into “*ER doctors can read patient’s medical history.*”. For back translation, we translated each NLACP into German and then back into English using [71]. These techniques introduced lexical variation while largely preserving sentence structure, improving LM generalization.

We manually annotated the augmented data with six ACR components: subject, action, resource, purpose, condition, and access decision. Missing components were marked as *none*. For NLACP identification, synthetic non-NLACP and NLACP sentences were labeled as 0 and 1, respectively, while preserving the original sentence-type annotations from Slankas *et al.*’s dataset. The lead author performed the initial annotation. Then, two cybersecurity researchers familiar with access control independently annotated a randomly selected 20% subset. The resulting Cohen’s kappa score ($\kappa = 0.76$) indicated substantial agreement [72]. All disagreements were resolved through discussion, and the full dataset was refined accordingly.

For LLM-based synthetic data generation, we used OpenAI’s GPT-4 [73] with in-context learning. We designed prompts to generate synthetic natural-language access control requirements and their entity annotations for given domains, such as healthcare and education, according to our access control policy definition. Few-shot examples were included to improve generation quality [58].

```

1 You are a system administrator working in the <
  domain> domain who is able to generate domain-
  related natural language access control
  requirements used to control access of users
  to resources in an organization. Generate such
  a natural language access control requirement
  and generate its structured representation
  according to the provided access control
  policy definition below. Output the natural
  language policy and its representation
  according to the following format without
  providing any additional text.
2
3 <Access control policy definition>
4 <few shot example 1>
5 <few shot example 2>
6 <few shot example 3>
7
8 Policy: <natural language access control policy>
9 ACP: <structured representation>

```

After generating the synthetic data, we manually reviewed each sentence and its annotation to accept, reject, or refine them if necessary. It is important to emphasize that we only use such a commercialized black-box LLM to generate *synthetic* data for fine-tuning our LMs. We do not use it for the policy generation (Section 3.4) that deals with organization’s confidential real-world access control requirements as it requires sending them to a private entity that controls GPT-4 exclusively, making the organization prone to data leakages [23] (Section 3.4).

After preparing the access control policy dataset (Section 4.1), we convert each annotated sentence into training examples with three roles: *system* which provides instructions to perform a particular task (Line 1-5), *user* who provides the input containing the NLACP and relevant information that can be used to translate the NLACP (Line 7-10), and *assistant* who provides the answer to the user query according to the system instructions (Line 12-14). Finally, we applied the following LLaMa 3 chat template to each training example during the training phase of the CHAGent’s access control policy generation module as advised by [74].

```

1 <|begin_of_text|><|start_header_id|>system<|
  end_header_id|>
2 <Access control policy definition>
3
4 Given a natural language sentence (i.e., NLACP),
  generate an access control policy according to
  the above Access Control Policy Definition.
  If a value for any key in any Python
  dictionary cannot be found in the NLACP, set
  the value to 'none'. To identify subject,
  resource, purpose, and condition, use the
  entities provided as a dictionary, 'Available
  entities' as an aid. If none of the provided
  Available entities match the entities of the
  NLACP or there are no 'Available entities'
  provided, use your judgment to select the most
  suitable entity within the NLACP
5 <|eot_id|>
6
7 <|start_header_id|>user<|end_header_id|>
8 NLACP: <NLACP>
9 Available entities: <relavant entities>
10 <|eot_id|>
11
12 <|start_header_id|>assistant<|end_header_id|>
13 <entity annotation>
14 <|eot_id|>

```

<|start_header_id|>x<|end_header_id|> is a special token sequence for LLaMa 3 LLM to indicate the starting points of each role *x* involved in the chat. *x* can be replaced with either “system”, “user” or “assistant”. Additionally, <|begin_of_text|> and <|eot_id|> indicate the beginning of the training example and the end of the turn of each role (i.e., system, user, or assistant), respectively.

4. Access Control Policy Refinement Dataset

As described in Section 4.3, the complete structure of a training example in the access control policy refinement dataset can be shown as follows. Similar to training examples from the access control policy dataset, we use the same

chat message template to create training examples. Finally, we applied the LLaMa 3 chat template to each training example (i.e., chat messages) during the training phase of the CHAGent’s access control policy generation module.

```

1 <|begin_of_text|><|start_header_id|>user<|
  end_header_id|>
2 <Access control policy definition>
3
4 You generated <incorrectly generated policy> for
  the sentence, <NLACP> based on the mentioned
  Access Control Policy Definition. However, the
  following error is found.
5
6 1. <error>
7
8 Please address the error and output the corrected
  access control policy according to the
  mentioned definition. Think step-by-step to
  first provide the reasoning steps/thought
  process. Then after the
9
10 ### Corrected:
11
12 provide the corrected policy without any other
  text.
13 <|eot_id|>
14
15 <|start_header_id|>assistant<|end_header_id|>
16 <reasoning according to the chain-of-thought>
17
18 ### Corrected:
19 <correct access control policy>
20 <|eot_id|>

```

It is important to note that, instead of providing the “correct access control policy” alone as the label (i.e., the response of the “assistant”) for the training example, we added the intermediate reasoning steps according to *chain-of-thought* [75], before providing the answer/label. It helps train the policy generation module to address the provided <error> step-by-step, outputting the intermediate steps that lead to the correct access control policy at the end. By doing so, the policy generation module based on LLaMa 3, which is pre-trained to predict the next token given the previous tokens (i.e., causal language modeling) [45], will be able to accurately generate the refined access control policy based on the refining steps, in the end, [75], [76].

5. Access Control Policy Validation

Figure 6 shows the confusion matrix of the best-performing seed to provide a more detailed view of the validator’s error patterns.

6. User Study Participant Demographics

A detailed breakdown of our user study participant demographics is shown in Table 8.

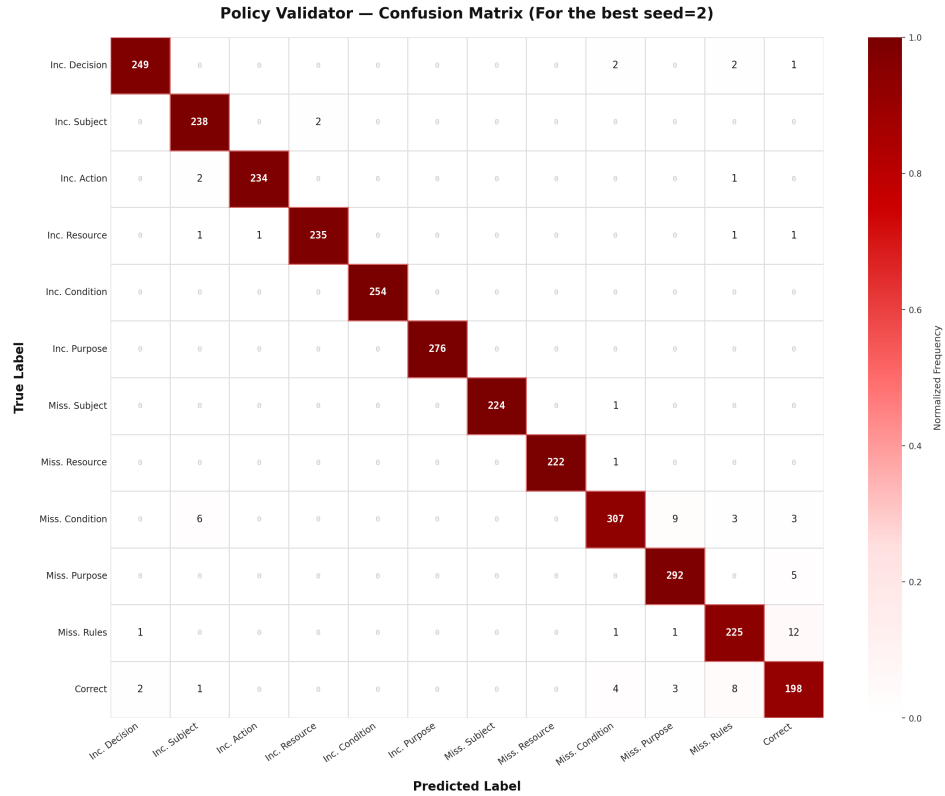


Figure 6: Confusion matrix of policy validator predictions for seed 2 ($F1_{macro} = 95\%$, Matthews Correlation Coefficient (MCC) = 0.9).

TABLE 8: Participant details. AC: Access Control, Org.: Organization, SE: Software Engineering. RBAC: Role-Based Access Control, ABAC: Attribute-Based Access Control, DAC: Discretionary Access Control, and MAC: Mandatory Access Control

ID	Age	Gender	Job Title	AC experience (years)	Familiar AC models	Org. Size	Org. Domain
P01	34	M	Software Engineer	2-5	RBAC, DAC, MAC	250-999	IT
P02	21	F	DevOps Engineer	2-5	RBAC, ABAC, DAC, MAC	50-249	IT
P1	26	M	DevOps Engineer	2-5	RBAC	50-249	Finance
P2	38	M	IT Officer	2-5	RBAC	250-999	Healthcare
P3	53	M	DevOps Engineer	2-5	RBAC	1000+	Military
P4	34	M	System Administrator	2-5	RBAC	1000+	IT
P5	64	M	CTO	>10	RBAC	50-249	IT
P6	38	M	IT Operations Analyst	2-5	RBAC, ABAC	>1000	IT
P7	22	M	Cybersecurity Specialist	2-5	RBAC, ABAC	1000+	Cybersecurity
P8	22	F	System Administrator	<1	RBAC	10-49	Telecommunication
P9	47	M	Cybersecurity Consultant	2-5	RBAC	50-249	Cybersecurity
P10	34	M	Cybersecurity Specialist	2-5	RBAC	50-249	Cybersecurity
P11	40	F	System Administrator	2-5	RBAC	250-999	Agriculture
P12	34	M	Security Team Leader	2-5	RBAC	>1000	Finance
P13	34	M	Head of Products	>10	RBAC, ABAC, DAC	10-49	IT
P14	49	F	IT Operations Analyst	6-10	RBAC	>1000	Finance
P15	37	M	Head of Products	6-10	RBAC, ABAC	10-49	Tourism
P16	30	M	Head of SE	<1	RBAC, ABAC	1-9	Finance
P17	41	M	Cybersecurity Consultant	2-5	RBAC	>1000	IT